

PRE-DEFENSE SUPERVISOR BRIEFING

AEGIS: A Constraint-Based Architecture for Safe Natural-Language Database Reporting

Research Gap · Methodology · Novelty · Defense Q&A Preparation

Prepared by: Md. Riaz

Programme: B.Sc. in CSE, Pundra University of Science and Technology

Purpose: Briefing document for thesis supervisor ahead of the thesis defense

Repository: github.com/md-riaz/AEGIS-Research

Date: July 2026 (revised to reflect a 31-source literature review)

How to Read This Document

This briefing is written for you, my supervisor, to review before the defense. It is deliberately organized the way a defense board expects to see a thesis explained: what was missing in prior work, what was actually studied and how, what is genuinely new, and — because the board will ask hard questions — a rehearsal set of likely questions with the answers I plan to give.

It does not describe the software prototype step by step. The prototype is only the instrument I built to test a research idea; the five sections below describe the research itself.

Section	What it answers
1. Research Gap	What did 31 related sources already solve, and what did none of them solve?
2. Methodology	What did I actually investigate, and how was the investigation carried out?
3. Novelty	What is genuinely new about the way I framed and answered the question?
4. Practical Use Case	Who actually uses this after deployment, and what does it look like day to day?
5. Defense Q&A Prep	What is the committee likely to ask, and how do I answer without overclaiming?

1. Research Gap Analysis

1.1 What the existing literature already covers

A systematic re-check of this thesis's full reference collection against the source PDFs (undertaken while drafting the thesis book's literature review chapter) found that this briefing's earlier count of 21 papers undercounted the collection, and that two files had been filed under the wrong name. The corrected, expanded review now covers **31 sources** spanning six research clusters: traditional natural-language database interfaces, neural and LLM-based text-to-SQL, conversational/contextual querying, applied conversational business intelligence, natural-language-to-visualization/dashboard generation, and AI-dashboard adoption & governance. Two citations are corrected from the earlier draft: the file labeled “Su et al.” is the genuine TriSQL paper (Su et al., 2026, Scientific Reports 16:7892), and the file labeled “Shailesh et al.” is the genuine Conversational BI paper (Shailesh et al., 2025, IJERT 14(12)) — both were re-verified directly against the source PDFs and are cited below under their correct content.

Cluster	Representative work	What it contributes / where it stops
NLIDB surveys & practice	Affolter et al.; Li & Jagadish (NaLIR); Lehmann et al.; Liu & Xu (2025 review)	Compare or design interaction methods for query expressiveness; Liu & Xu is the only one to name SQL-injection risk (TrojanSQL) directly, without proposing an architectural defense.
Neural / LLM text-to-SQL	Seq2SQL; RAT-SQL; PICARD; G-SQL; TriSQL; Spider; BIRD; Jha & Anand (2025); Pinna et al. (2025, evaluation metrics)	Improve SQL accuracy, schema linking, or syntactic validity — but the model still authors the executable query. TriSQL (2026) is the current state of the art (82.2% execution accuracy on Spider) and still has the LLM produce the final SQL directly, with no reported safety evaluation.
Conversational / contextual	SParC; CoSQL	Handle ambiguity and follow-up turns, but still target generated SQL as the end product, with no persistence layer.
Applied conversational BI	Shailesh et al. (2025); Mujeeb et al. (2025); Chinnappaian (2025)	Closest real-world comparisons. Shailesh et al. is a working Groq/LangChain assistant that gives the LLM direct SQL execution tools and a self-correction loop — a live example of the exact attack surface this thesis's threat model closes. None define a formal threat model, a deterministic compiler, or a closed business vocabulary.
NL-to-visualization / dashboards	DataTone; NL4DV; nvBench; DashBot; Kavaz et al. (2023)	Solve chart selection or dashboard composition (or survey that sub-field) assuming the data access problem is already solved elsewhere.
AI-dashboard adoption & governance	Häikiö (2024); Saidur (2025); Valkenburgh (2024)	Business/IS-governance and adoption literature, mostly adjacent to the technical contribution. Valkenburgh is the exception: an independently-arrived-at “let a deterministic layer compute the answer, let the model only narrate it” design, in an unrelated domain (spreadsheet-based explanatory analytics).

1.2 The five gaps that remain unresolved

Gap 1 — The language model still authors executable SQL

Even constrained-decoding methods like PICARD, or schema-aware methods like RAT-SQL, G-SQL and the state-of-the-art TriSQL (2026), only make model-generated SQL more accurate or syntactically valid. The probabilistic model is still the one constructing the query, so a syntactically perfect statement can still use the wrong business formula, touch an unapproved join, or return unauthorized rows.

Gap 2 — A database schema does not encode business meaning

Schema-linking work connects words to tables and columns, but terms like “revenue,” “active customer,” or “refund rate” are organization-specific definitions, not schema facts. No reviewed system inserts an explicit, administrator-approved semantic layer between language understanding and query compilation.

Gap 3 — Safety is checked after the SQL already exists, not before

None of the reviewed systems treat the authenticated user's role as an input to query construction, or define a formal family of permitted queries per role. Shailesh et al.'s deployed assistant illustrates this concretely: it gives the LLM a SQL-execution tool and a self-correction loop, with no reported adversarial evaluation. Benchmark execution accuracy therefore cannot be read as evidence of authorized access.

Gap 4 — No single architecture connects all the pipeline stages

Text-to-SQL systems stop at SQL. NL4DV, DataTone and nvBench stop at a chart specification. DashBot stops at dashboard layout. Nothing in the literature connects intent understanding, business-semantic mapping, authorized compilation, visualization choice, and artifact persistence in one controlled pipeline.

Gap 5 — Every request is treated as disposable

Almost all reviewed systems answer a question once and discard the analysis. None convert a recognized, recurring reporting need into a versioned, refreshable, access-controlled widget.

1.3 Gap statement

Existing natural-language database research focuses on making model-generated SQL more accurate or more syntactically valid — including the most recent state-of-the-art systems. It does not investigate an alternative in which the language model is prevented from generating executable SQL altogether — where the LLM is restricted to analytical-intent extraction, and business definitions, authorization rules, and executable SQL are all produced deterministically by a compiler operating over an administrator-approved semantic layer.

2. Research Methodology

2.1 What is actually being investigated

This is not a study about building a better text-to-SQL tool. It is a study of one architectural decision and its consequences: who is allowed to author the executable SQL — the language model, or a deterministic compiler?

Element	Description
Independent variable	Who authors the SQL — the LLM (baseline) or a deterministic compiler (proposed method)
Dependent variables	Unsafe-query rate, execution validity, intent accuracy, business-metric correctness, join correctness, authorization correctness, request coverage, latency
Research question	Does separating probabilistic language understanding from deterministic SQL compilation improve safety and semantic consistency compared with direct LLM-to-SQL generation?
Hypothesis	Restricting the LLM to intent extraction and generating SQL deterministically from an approved semantic layer reduces unsafe and semantically incorrect queries relative to direct LLM-to-SQL generation.

2.2 The two architectures under comparison

Both conditions use the same model, the same database, and the same benchmark questions — only the location of SQL authorship changes. **Approach A (baseline):** NL question → LLM understands request and generates SQL itself → Database → Query result. **Approach B (proposed):** NL question → LLM extracts analytical intent only → Validate against approved concepts → Deterministic compiler writes the SQL → Database → Result + chart + widget.

The only structural difference is who is permitted to touch the SQL. In Approach A, natural language can influence the executable query's structure directly. In Approach B, natural language can influence only a typed intent object built from a closed vocabulary; the SQL structure, joins, and permission predicates all come from trusted, administrator-controlled definitions.

2.3 The seven-stage AEGIS pipeline (proposed method)

1. Intent extraction — LLM converts NL request into typed JSON
2. Validation — reject identifiers outside approved vocabulary
3. Semantic mapping — business terms → approved SQL definitions
4. Permission rewriting — role predicates added outside LLM control
5. Deterministic compilation — parameterized read-only SQL is built
6. Visualization selection — chart chosen from intent + result shape
7. Widget persistence — plan, query identity & policy saved for reuse

A post-compilation validator additionally accepts only a single read-only SELECT statement and rejects any forbidden construct. The threat boundary explicitly assumes the semantic-layer definitions, permission rules, and compiler templates themselves are trusted, administrator-controlled resources — this is stated up front rather than left implicit, because a committee member may otherwise assume the architecture claims to defend against a malicious administrator, which it does not.

2.4 One request, start to finish

Trace for “Show revenue by category this month”: **Stage 1** — LLM returns {intent_class: ranking, metric_term: revenue, dimension_term: category, time_term: this month, sort: desc, limit: 10} with no SQL field. **Stage 2** — both terms resolve against the approved vocabulary. **Stage 3** — revenue → SUM(oi.UnitPriceExcITax * oi.Quantity); category → c.Name, with their required join tables. **Stage 4** — a role predicate is appended using the session, never anything the user typed. **Stage 5** — BFS finds the shortest join path (Order → OrderItem → Product → Product_Category_Mapping → Category); a safety scan runs before execution. **Stage 6** — ranking + descending sort → horizontal bar chart. **Stage 7** — the plan is hashed (SHA-256) and saved as a refreshable widget. The words the user actually typed never appear in the compiled SQL string.

2.5 Three linked studies, not one

Study	What it investigates	Purpose
Study 1 — Reporting-pattern analysis	Whether common organizational reporting requests can be represented by a small, finite set of analytical patterns (KPI, ranking, trend, comparison, exception, summary, segment, funnel, cohort, correlation, tabular)	Establishes that a bounded vocabulary is a reasonable design choice, not an arbitrary restriction
Study 2 — Architectural comparison	Direct LLM-to-SQL vs. AEGIS's constraint-based compilation, same model & database, on a 100-question benchmark covering supported, paraphrased, unsupported, and adversarial requests	Tests the central hypothesis: does removing SQL authorship from the LLM measurably reduce unsafe/incorrect queries?
Study 3 — Cross-schema transfer	The same architecture reconfigured for a second schema (WooCommerce) by replacing only the semantic-layer definitions, evaluated on a second question set	Tests whether the contribution is a general method or a one-off solution tied to a single database

2.6 Evaluation metrics

Dimension	Metric
Intent understanding	Precision, recall, F1, exact slot accuracy
Semantic correctness	Agreement with the approved metric formula and join path
Execution	Executable-query rate, result equivalence
Safety	Unsafe-query rate, adversarial rejection rate
Authorization	Forbidden-table / column / row access rate
Coverage	Direct / clarified / extended / rejected proportions
Visualization	Agreement with expert-approved chart choice
Portability	Configuration time and code changes for the second schema
Performance	Median / p95 latency per pipeline stage

Safety and authorization are scored separately, deliberately: a query can be syntactically safe and still violate a role's access policy, or compute the wrong business number while executing without error.

3. Research Novelty

AEGIS does not introduce a new LLM, SQL parser, charting library, or database engine. The contribution is the research formulation and the architectural boundary — and the evaluation of whether that boundary helps.

Novelty 1 — Reformulating the target problem

Prior work treats the problem as Natural Language → SQL. This research reframes it as Natural Language → Approved Analytical Intent → Deterministically Compiled Report. The object being generated by the model changes from an executable instruction to a bounded, checkable description of intent.

Novelty 2 — Testing whether a bounded reporting vocabulary is viable

Instead of assuming organizational reporting needs unrestricted SQL expressiveness, the research empirically tests whether a small, finite set of analytical patterns can cover most real reporting requests — trading some expressiveness for safety, consistency, and auditability.

Novelty 3 — A hard boundary between probabilistic and deterministic responsibility

LLM-controlled: interpreting user intent, identifying the reporting pattern, extracting filter values, flagging ambiguity. System-controlled: business metric & dimension definitions, table and join selection, permission/role predicates, final SQL structure. This is different from prompting an LLM to “please generate only safe SQL.” Here, safety is a property of the architecture, not an expectation about model behavior.

Novelty 4 — A formal, role-dependent safe query space

$q \in Q_safe(L, r)$, where L is the approved semantic-layer definitions and r is the authenticated user's role. A generated query is only accepted if it uses an approved analytical pattern and metric/dimension, follows an approved join path, includes the required permission restriction, performs only read operations, and passes literal values as bound parameters — not if it merely executes without error.

Novelty 5 — Separating “SQL correctness” from “business correctness”

A query can execute successfully and still be wrong: it might use an unapproved revenue formula, include cancelled orders it shouldn't, or expose rows outside the requester's permission. This research treats business-semantic correctness as a distinct, separately measured property rather than folding it into execution accuracy.

Novelty 6 — Turning one-off questions into reusable artifacts

Differently worded requests that mean the same analysis are converted into the same standard definition and stored as a refreshable widget, rather than being answered and discarded each time.

One-paragraph novelty summary for slides

The contribution is not another text-to-SQL system. It is the reformulation of natural-language reporting as intent-to-report compilation, and the empirical test of whether structurally removing SQL authorship from the language model — while keeping business definitions, permissions, and query structure under deterministic control — produces safer and more semantically consistent reporting than direct LLM-to-SQL generation, without an unacceptable loss of coverage.

One-minute spoken explanation

“My research investigates an architectural question in natural-language database reporting. Existing work mainly improves the accuracy or syntax of SQL generated by language models — including the current state of the art. I investigated whether the model could instead be restricted to understanding the user's analytical intention only, while executable SQL is produced deterministically from approved business definitions and authorization rules. I compared this constraint-based architecture against direct LLM-to-SQL under the same model, database, and questions, measuring safety, semantic correctness, authorization, coverage, and latency. The contribution is not the prototype — it is the formulation and evaluation of intent-to-report compilation as an alternative to direct LLM-generated SQL.”

4. Practical Use Case — Life After Deployment

A defense committee will eventually ask “who actually uses this, and why does it matter?” The answer is that the permission model already built into AEGIS — five roles enforced at the compiler stage — maps directly onto how a real retail or e-commerce organization is structured.

AEGIS role	Typical job title	Typical question asked	What they actually see
store_manager	Store / branch manager	“How much did we sell today?”	Only rows belonging to their own store — a permission predicate is appended after the LLM, using their session, not their wording.
regional_manager	Area / regional manager	“Compare revenue across my stores this month”	Rows for every store in their region — a wider but still bounded slice of the same data.
analyst	Business / data analyst	“Refund rate by category last quarter”	Broad read access across the analytics domain, still limited to the approved metric and dimension vocabulary.
read_only	Executive / internal auditor	Opens existing saved dashboards	Can view and refresh saved widgets but cannot trigger new query compilation.
public	Unauthenticated / demo user	Sample questions during a demo	Only non-sensitive, pre-approved aggregate metrics.

4.1 A day in the life

A store manager opens the dashboard each morning and asks “How did we do yesterday?” — this becomes a saved KPI widget that refreshes automatically every morning. A regional manager asks a related but broader question and gets a ranking widget scoped to their whole region, from the same semantic layer and compiler, just a different permission scope. An analyst investigating a margin drop builds a deeper correlation widget that the rest of the team reuses instead of rebuilding. None of these three people write SQL, touch a shared query editor, or can see another role’s data by construction rather than by convention.

4.2 Why this matters for the thesis argument

This isn’t a feature bolted on after the fact — it’s the same permission-rewriting stage (Stage 4) and the same widget-persistence stage (Stage 7) already evaluated in the manuscript. The practical value of “safe by construction” and “reusable by default” is that an organization’s existing reporting hierarchy — store, region, head office — maps onto AEGIS roles without any custom development.

5. Defense Q&A Preparation

Grouped by the angle a committee member is likely to attack from. My planned answers are underneath each question.

On the research gap

Q: Isn't this just adding a permission check on top of existing text-to-SQL systems?

No. A permission check that runs after SQL is generated is a filter on an already-untrusted artifact. AEGIS never lets the LLM produce SQL in the first place — the permission rule is inserted during deterministic compilation, using the authenticated session, not anything derived from the user's wording.

Q: PICARD (or TriSQL) already constrains SQL generation — how is this different?

PICARD rejects syntactically invalid tokens during decoding; TriSQL adds schema selection, skeleton-first decoding, and an LLM refinement loop. Both raise accuracy, but the LLM still writes the final SQL string in both cases, with no reported safety/injection evaluation. AEGIS removes SQL authorship from the model entirely, so the question of what the model might write does not arise the same way.

Q: Why not just fine-tune or prompt-engineer the LLM to always generate safe SQL?

Because that makes safety a property of model behavior, which is probabilistic and cannot be exhaustively verified. AEGIS makes safety a property of the architecture: the LLM structurally cannot emit SQL tokens, tables, or joins — its output is a typed object with no SQL field.

On methodology and evidence

Q: Why only compare against direct LLM-to-SQL? What about other baselines?

The design also defines decomposed LLM-to-SQL, keyword-to-template matching, and four AEGIS ablations (no vocabulary injection, no semantic layer, no permission rewriting, no post-compilation validation) as comparison points, to show which component is responsible for which improvement.

Q: Isn't a 100-question benchmark small?

It is intentionally exhaustive rather than large: constructed to cover every supported analytical pattern plus paraphrases, unsupported requests, and adversarial/unauthorized requests, so safety and rejection behavior can be measured on purpose-built cases.

Q: How do you know the improvement isn't just a better prompt for the baseline?

Both conditions use the same underlying model, the same database snapshot, and the same benchmark questions; only the location of SQL authorship changes.

On the architecture itself

Q: What happens if the semantic layer itself is wrong or misconfigured?

The threat boundary explicitly assumes the semantic layer, permission rules, and compiler templates are trusted, administrator-controlled resources — that is a stated limitation, not a hidden one.

Q: What if the LLM's intent extraction is wrong — doesn't that still cause an incorrect report?

Yes, and that is why intent accuracy is measured as its own metric rather than assumed. A wrong intent produces a wrong-but-safe report; it will never produce an unauthorized or structurally unsafe one.

Q: Does the constrained vocabulary mean AEGIS can't answer many real questions?

Coverage is measured explicitly, with four categories: answered directly, answered after clarification, answered via an extended pattern, or explicitly rejected.

On practical use

Q: Who would actually use this after deployment?

The five roles already enforced by the permission-rewriting stage map directly onto a real organizational hierarchy: store manager, regional manager, analyst, read-only, and public.

On generalization

Q: Is this only built for nopCommerce?

The cross-schema study reconfigures the same architecture for WooCommerce by replacing only the semantic-layer definitions, keeping the intent model, pattern taxonomy, and safety mechanism unchanged.

On novelty

Q: Isn't "don't let the AI write SQL" an obvious idea?

The idea of constraining an LLM is not new by itself — PICARD, G-SQL, and the state-of-the-art TriSQL all constrain the LLM's output in some way. What is new here is going further: removing SQL authorship from the LLM entirely, clearly defining the resulting safe query space as a function of the semantic layer and the user's role, and empirically testing whether that stricter boundary is viable for real reporting workloads without collapsing coverage.

Q: What is the single sentence I should say if I only get one chance?

"I reformulated natural-language reporting from 'language model generates SQL' to 'language model extracts intent, deterministic compiler generates SQL,' and I tested whether that boundary improves safety and semantic correctness without an unacceptable loss of coverage."

Prepared as a study aid ahead of the defense. All figures and dataset sizes referenced here should be reconciled with the final manuscript before submission.